

Práctica 2: Aprendizaje automático para la lectura de paneles informativos en autopistas.

ALBERTO FIERRO LEVA

JUAN GÓMEZ REY

SERGIO MUÑOZ LAUREIRO

Ejecución del programa:	2
Ejercicio1: Entrenamiento y validación del clasificador de caracteres	3
Ejercicio 2. Probar otras alternativas para el reconocimiento	12
Ejercicio 3: Lectura automática de paneles recortados	12
Ejercicio 4: Lectura automática de paneles	17

Introducción

Esta es la segunda práctica de la asignatura “Visión artificial” de la carrera de Ingeniería informática en la URJC. El objetivo es el reconocimiento de texto de carteles de tráfico azules típicos de autovías, para ello se ha creado un programa en python que reconozca los caracteres.

Esta práctica es una ampliación de la primera, se contará con la detección y extracción de caracteres, entrenamiento de datos y reconocimiento.

Uso

Para ejecutar el programa se pueden usar este comando en el directorio raíz, la segunda versión sin visualización OCR:

```
python main.py --test_path Practica1/test_detection --train_ocr train_ocr
```

```
python main.py --test_path Practica1/test_detection --train_ocr train_ocr --visualize_ocr
```

Ejercicio1: Entrenamiento y validación del clasificador de caracteres

Este ejercicio se resolverá principalmente en el archivo `lda_normal_bayes_classifier.py`

Cargar y umbralizar las imágenes de los caracteres de entrenamiento:

Esta parte se resuelve en `_extraer_caracteristicas()`:

Se aplica un umbralizado adaptativo y tras eso se busca el contorno mas grande (que asumimos que es la letra y recortamos la imagen).

```
# 1. Asegurar escala de grises
if len(img.shape) == 3:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
else:
    gray = img.copy()

# 2. Umbralización adaptativa (las letras son claras sobre fondo
# oscuro o viceversa)
thresh = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, 11, 2
)

# 3. Buscar el contorno del carácter
contornos, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)

if contornos:
    # Asumimos que el contorno más grande es la letra
```

```

c = max(contornos, key=cv2.contourArea)
x, y, w, h = cv2.boundingRect(c)
recorte = thresh[y:y+h, x:x+w]
else:
    # Si falla, usamos la imagen original
    recorte = thresh

```

Construir el vector de características a partir de las imágenes de entrenamiento del carácter ya recortadas usando el rectángulo detectado.

Esta parte también se resuelve en `_extraer_caracteristicas()`:

Se redimensiona la imagen a un tamaño de 25x25 y se aplanar a una dimension.

```

# 4. Redimensionar a tamaño fijo (25x25)
resized = cv2.resize(recorte, self.ocr_char_size)

# 5. Aplanar a un vector de 1 dimensión (tamaño 625)
vector_caracteristicas = resized.flatten().astype(np.float32) /
255.0

return vector_caracteristicas

```

Reducir la dimensión de los vectores de características usando LDA (Linear Discriminant Analysis).

Este ejercicio se resuelve en `train()`:

Mediante un for loop se generan las matrices características y etiquetas, tras este primer paso ya se puede empezar a entrenar, usaremos LDA y un clasificador bayesiano, también incluidos en la función `train()`:

```

def train(self, images_dict):
    """
    Given character images in a dictionary of list of char images of
    fixed size,
    train the OCR classifier. The dictionary keys are the class of
    the list of images
    (or corresponding char).

    :images_dict is a dictionary of images (name of the images is the
    key)
    """

    # Take training images and do feature extraction

```

```

X = [] # Feature vectors by rows
y = [] # Labels for each row in X

# 1. Construir las matrices C (características) y E (etiquetas)
for char_key in sorted(images_dict.keys()):
    list_images = images_dict[char_key]
    label = self.char2label(char_key)
    for img in list_images:
        features = self._extraer_caracteristicas(img).astype(np.float64)
        X.append(features)
        y.append(label)

# Scikit-learn acepta float64 (por defecto en np), pero
preparamos el array
samples = np.array(X, dtype=np.float64)
labels = np.array(y, dtype=np.int32)

# Perform LDA training

print("Entrenando proyector LDA...")
# 2. Perform LDA training and dimension reduction (Generar
Matriz CR)
X_reduced = self.lda.fit_transform(samples, labels)

# Perform Classifier training

print("Entrenando clasificador Bayesiano Normal...")
# 3. Perform Classifier training (OpenCV ml)
X_reduced_cv = np.array(X_reduced, dtype=np.float32)

self.classifier.train(X_reduced_cv, cv2.ml.ROW_SAMPLE, labels)

return samples, labels

```

Para hacer uso del clasificador se hace escribiendo este comando:

```
python evaluar_clasificadores_OCR.py --classifier lda_bayes
```

En evaluar_clasificadores_ocr.py se añade el siguiente código para poder utilizar el LDANormalBayesClassifier:

```

# 1) Cargar imágenes de entrenamiento
print("Cargando datos de entrenamiento...")
train_dict = cargar_diccionario_imagenes(args.train_path)
print(f" Clases encontradas: {len(train_dict)}")

# 2) Cargar imágenes de validación
print("Cargando datos de validación...")
val_dict = cargar_diccionario_imagenes(args.validation_path)

```

```
# 3) Crear y entrenar clasificador
if args.classifier == "lda_bayes":
    clf = LdaNormalBayesClassifier(ocr_char_size=(25, 25))
```

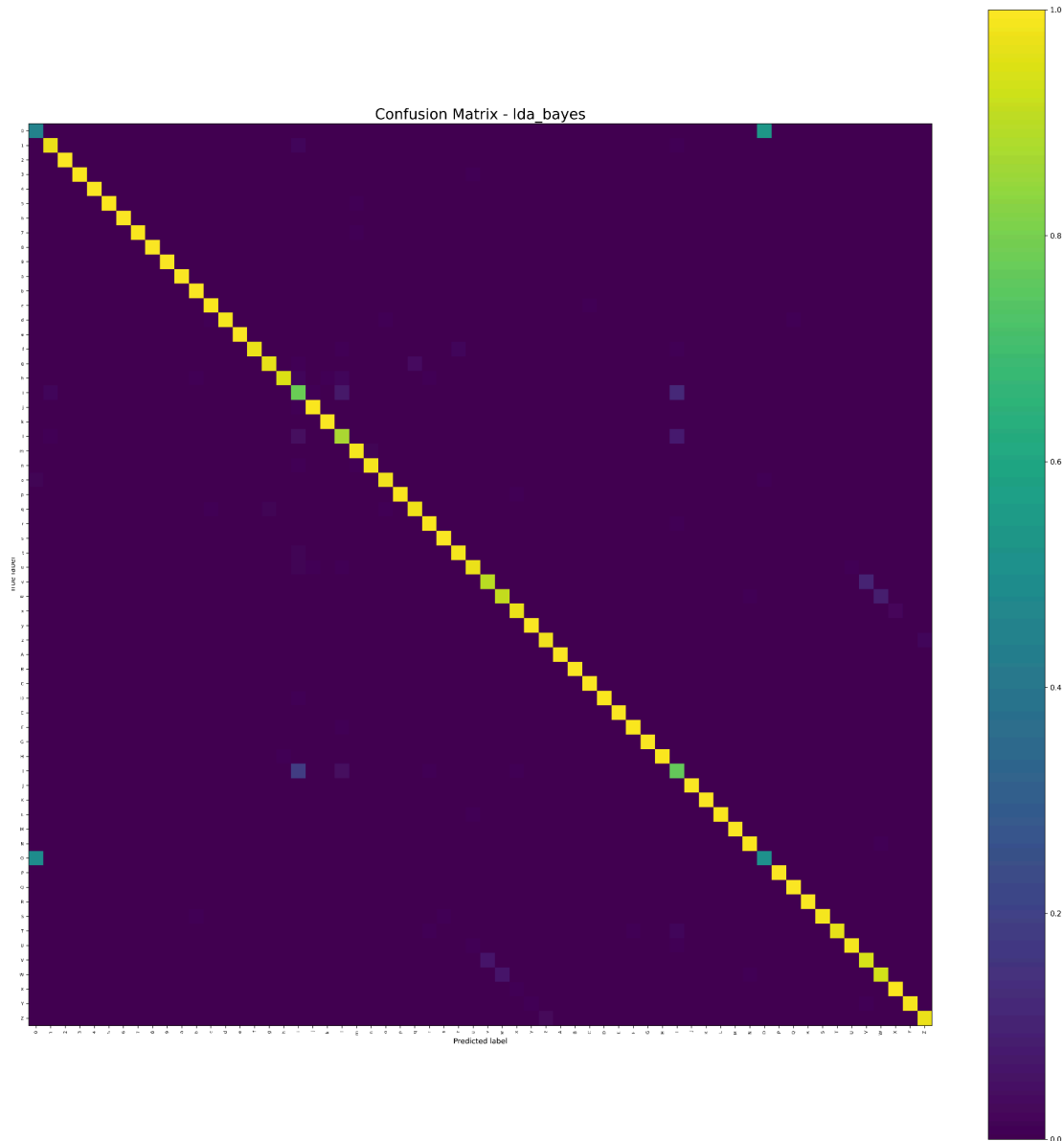


Figura 1: lda_bayes

Se observa en la matriz de confusión que los caracteres más problemáticos son el 0 y el 9.

Ejercicio 2. Probar otras alternativas para el reconocimiento

Para esta fase hemos probado otras tres alternativas:

- PCA-KNN
- PCA-Bayes

- HoG-svm

Para esto se han creado varios archivos:

- pca_knn_classifier.py
- pca_bayes_classifier.py
- train_hog_svm.py, hog_svm_classifier.py

PCA-KNN

```
class PcaKnnClassifier(OCRClassifier):

    def __init__(self, ocr_char_size=(25,25), n_components=50,
k=3):
        super().__init__(ocr_char_size)
        self.pca = PCA(n_components=n_components)
        self.knn = cv2.ml.KNearest_create()
        self.classifier_name = "pca_knn"
        self.k = k

    def train(self, images_dict):
        C = []
        E = []

        for key in sorted(images_dict.keys()):
            for img in images_dict[key]:
                feat = self._extraer_caracteristicas(img)
                C.append(feat)
                E.append(self.char2label(key))

        C = np.array(C, dtype=np.float32)
        E = np.array(E, dtype=np.int32)

        # PCA
        C_reduced = self.pca.fit_transform(C).astype(np.float32)

        # Entrenar KNN
        self.knn.train(C_reduced, cv2.ml.ROW_SAMPLE, E)

    def predict(self, img):
        feat = self._extraer_caracteristicas(img).reshape(1, -1)
        feat_pca = self.pca.transform(feat).astype(np.float32)

        ret, results, neighbours, dist =
self.knn.findNearest(feat_pca, self.k)
        return int(results[0][0])
```

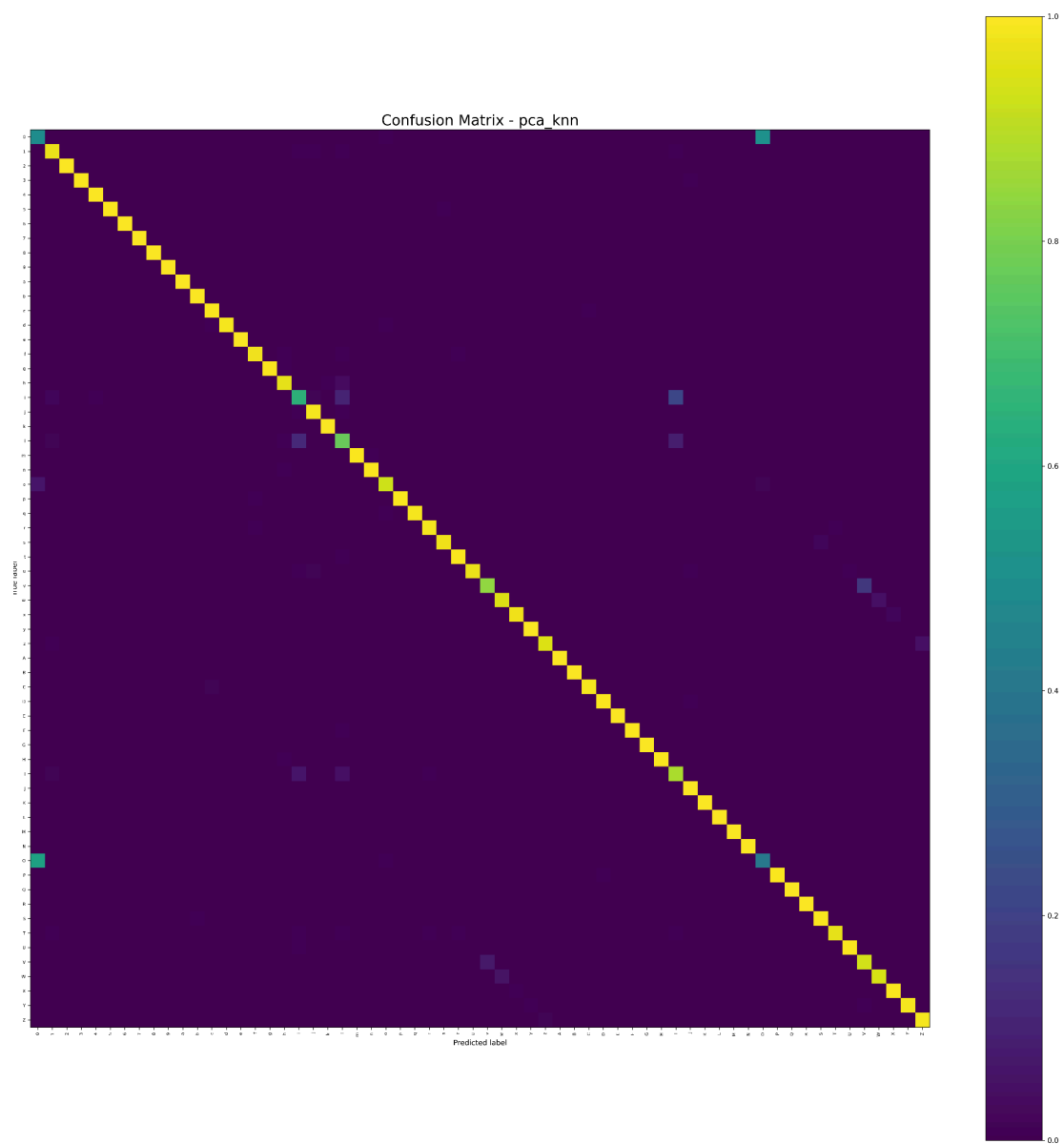


Figura 2:Matriz de confusión pca_knn

PCA-Bayes

```
class PcaBayesClassifier(OCRCClassifier):

    def __init__(self, ocr_char_size=(25,25), n_components=50):
        super().__init__(ocr_char_size)
        self.pca = PCA(n_components=n_components)
        self.bayes = cv2.ml.NormalBayesClassifier_create()
        self.classifier_name = "pca_bayes"

    def train(self, images_dict):
        C = []
        E = []

        for key in sorted(images_dict.keys()):
            for img in images_dict[key]:
                feat = self._extraer_caracteristicas(img)
                C.append(feat)
                E.append(self.char2label(key))

        C = np.array(C, dtype=np.float32)
        E = np.array(E, dtype=np.int32)

        # PCA
        C_reduced = self.pca.fit_transform(C).astype(np.float32)

        # Entrenar Bayes
        self.bayes.train(C_reduced, cv2.ml.ROW_SAMPLE, E)

    def predict(self, img):
        feat = self._extraer_caracteristicas(img).reshape(1, -1)
        feat_pca = self.pca.transform(feat).astype(np.float32)

        ret, pred = self.bayes.predict(feat_pca)
        return int(pred[0][0])
```

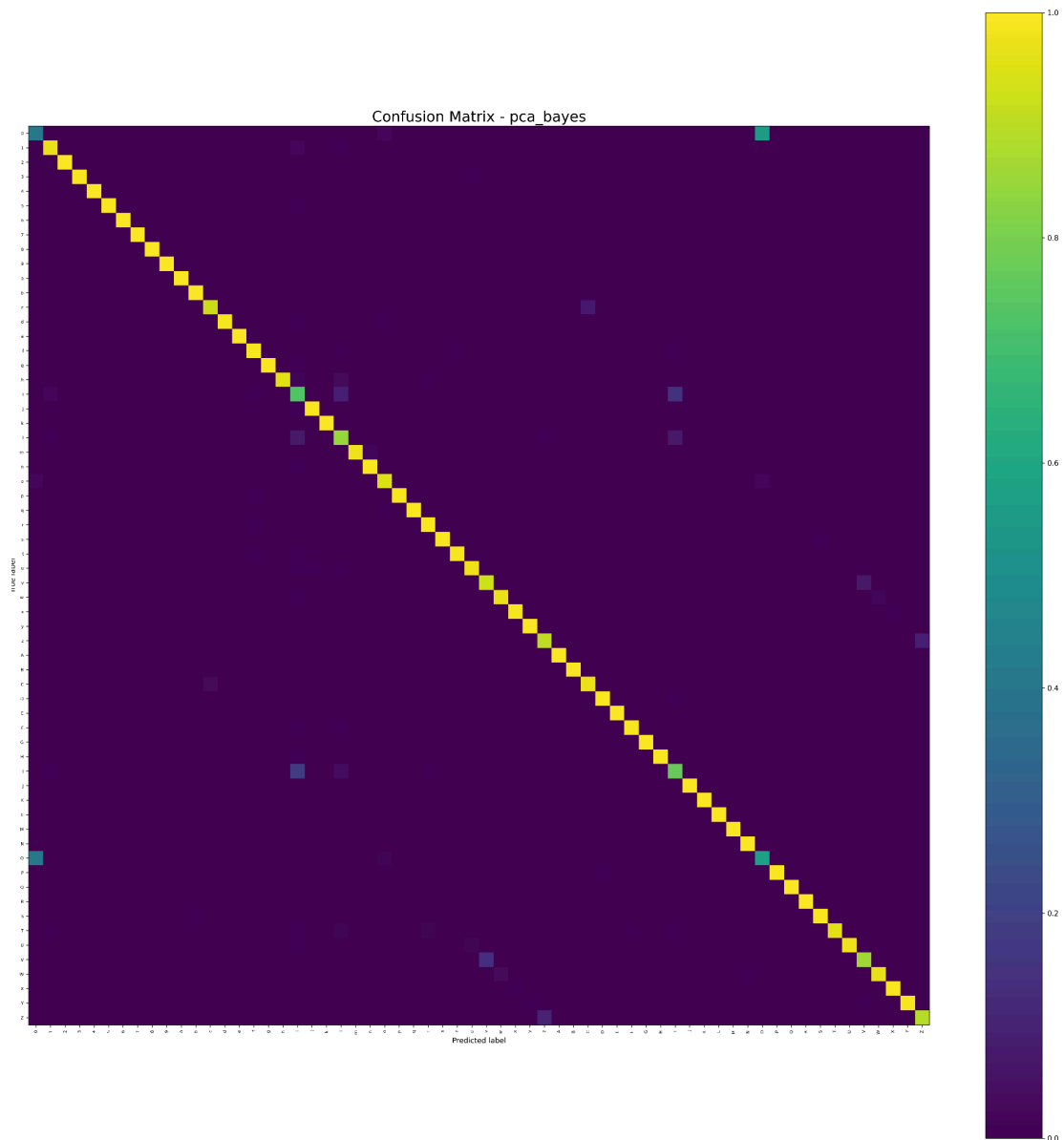



Figura 3: Matriz de confusión pca_bayes

HoG-svm

```
class HogSvmClassifier(OCRClassifier):

    def __init__(self, ocr_char_size=(25,25)):
        super().__init__(ocr_char_size)
        self.svm = cv2.ml.SVM_create()
        self.svm.setKernel(cv2.ml.SVM_RBF)
        self.svm.setType(cv2.ml.SVM_C_SVC)
        self.classifier_name = "hog_svm"

    def _hog_features(self, img):
        if len(img.shape) == 3:
            img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```

        img = cv2.resize(img, self.ocr_char_size,
interpolation=cv2.INTER_AREA)

        img = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)[1]
        img = cv2.GaussianBlur(img, (3, 3), 0)
        img = cv2.equalizeHist(img)
        img = cv2.copyMakeBorder(img, 2, 2, 2, 2,
cv2.BORDER_CONSTANT, value=0)
        img = cv2.resize(img, self.ocr_char_size)

        features = hog(
            img,
            orientations=9,
            pixels_per_cell=(5, 5),
            cells_per_block=(1, 1),
            visualize=False
        )
        return features

def train(self, images_dict):
    C = []
    E = []

    for key in sorted(images_dict.keys()):
        for img in images_dict[key]:
            feat = self._hog_features(img)
            C.append(feat)
            E.append(self.char2label(key))

    C = np.array(C, dtype=np.float32)
    E = np.array(E, dtype=np.int32)

    self.svm.train(C, cv2.ml.ROW_SAMPLE, E)

def predict(self, img):
    feat = self._hog_features(img).reshape(1,
-1).astype(np.float32)
    ret, pred = self.svm.predict(feat)
    return int(pred[0][0])

def save(self, filename):
    self.svm.save(filename)

def load(self, filename):
    self.svm = cv2.ml.SVM_load(filename)

```

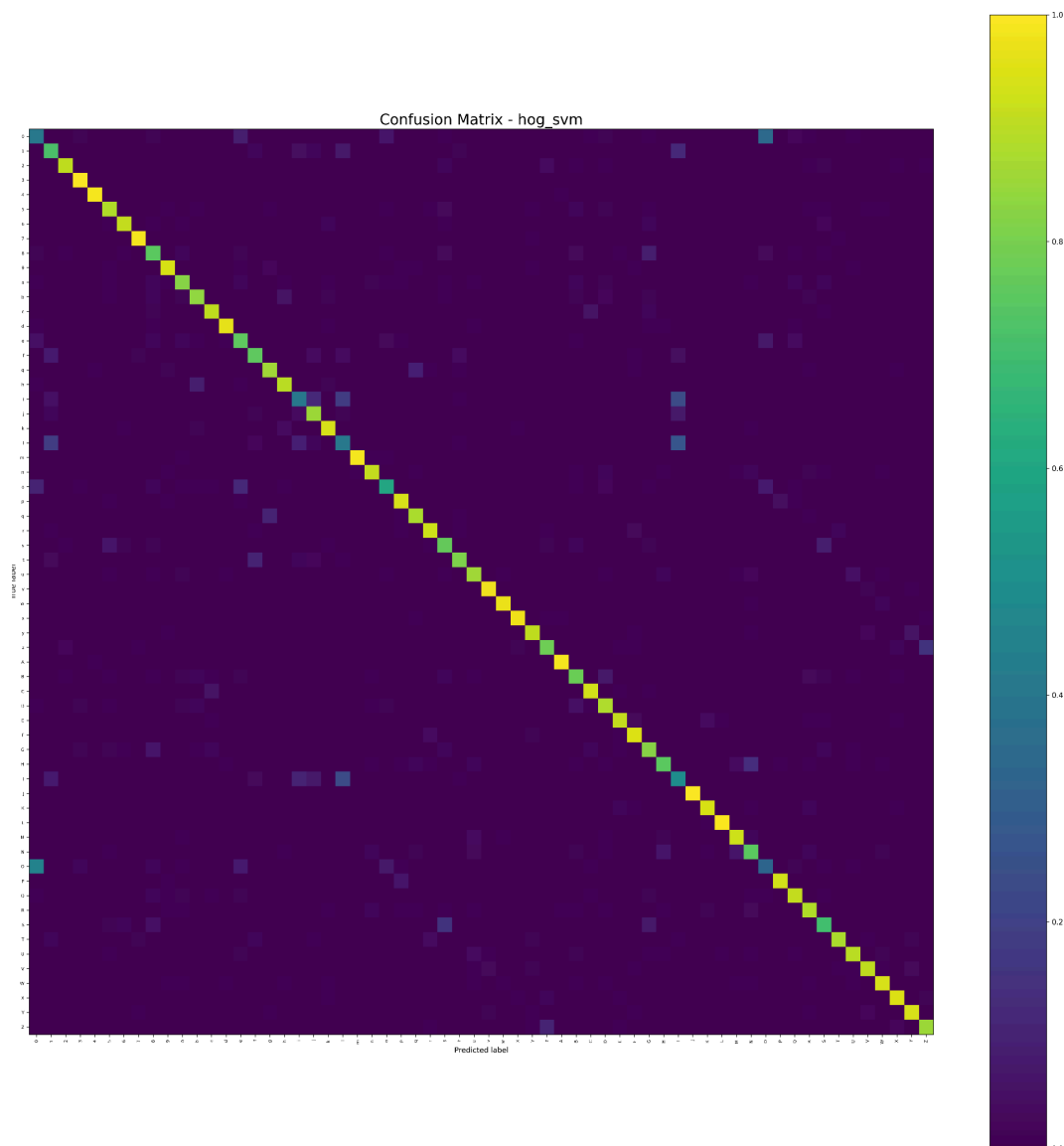


Figura 4:Matriz de confusión hog_svm

En la matriz de confusión se observa algo bastante esperado y es que el 'o' y el '0' e 'i' y 'l' se confunden entre sí bastante.

Se se pueden ejecutar estos clasificadores así:

- `python evaluar_clasificadores_OCR.py --classifier pca_knn`
- `python evaluar_clasificadores_OCR.py --classifier pca_bayes`
- `python evaluar_clasificadores_OCR.py --classifier hog_svm`

Además se ha añadido código en `evaluar_clasificadores_OCR.py` para poder usarlos.

```
elif args.classifier == "pca_knn":
    from pca_knn_classifier import PcaKnnClassifier
    clf = PcaKnnClassifier(ocr_char_size=(25, 25))
```

```
elif args.classifier == "pca_bayes":
    from pca_bayes_classifier import PcaBayesClassifier
    clf = PcaBayesClassifier(ocr_char_size=(25, 25))

elif args.classifier == "hog_svm":
    from hog_svm_classifier import HogSvmClassifier
    clf = HogSvmClassifier(ocr_char_size=(25, 25))
```

Comparación entre clasificadores:

Clasificador	Tasa de aciertos	Tiempo de entrenamiento	Tiempo medio de predicción
lda_bayes	0.96	2.05s	0.11ms
pca_knn	0.96	0.73s	0.62ms
pca_bayes	0.96	0.74s	0.096ms
hog_svm	0.85	76.8s	2.49ms

Los tres primeros son los que tienen mejor tasa de aciertos, hog_svm siendo el peor con diferencia. Los tres primeros solo tienen problema distinguiendo la o y 0 mientras que hog tiene problemas también con la i y con la l y con más caracteres puntuales (ver matriz de confusión hog_svm).

En cuanto a la eficiencia los dos mejores en cuanto a entrenamiento son pca_knn y pca_bayes. Los dos mejores en cuanto a tiempo de predicción son pca_bayes y lda_bayes. El mejor de forma general es pca_bayes y el peor de forma general es hog_svm.

Ejercicio 3: Lectura automática de paneles recortados

En este siguiente ejercicio tenemos que detectar de forma automática los caracteres de las señales de tráfico, todo en el archivo main_panels_ocr.py para ello hemos seguido:

1. Detectar los caracteres mediante umbralización y búsqueda de componente

```
def detectar_caracteres_panel(img):
    escala = 1.5
    img_scaled = cv2.resize(img, None, fx=escala, fy=escala,
                             interpolation=cv2.INTER_LINEAR)
    gray = cv2.cvtColor(img_scaled, cv2.COLOR_BGR2GRAY)
    h_img, w_img = gray.shape

    thresh_inv = cv2.adaptiveThreshold(
        gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY_INV, blockSize=11, C=6
    )
```

```

thresh_norm = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, blockSize=11, C=6
)

boxes_inv = detectar_boxes_from_thresh(thresh_inv, h_img,
w_img)
boxes_norm = detectar_boxes_from_thresh(thresh_norm,
h_img, w_img)

def score_boxes(boxes):
    if not boxes: return 0
    h_media = np.mean([bh for (bx,by,bw,bh) in boxes])
    return len(boxes) * h_media

if score_boxes(boxes_norm) > score_boxes(boxes_inv):
    boxes = boxes_norm
    polaridad = 'norm'
else:
    boxes = boxes_inv
    polaridad = 'inv'

return boxes, gray, img_scaled, polaridad

```

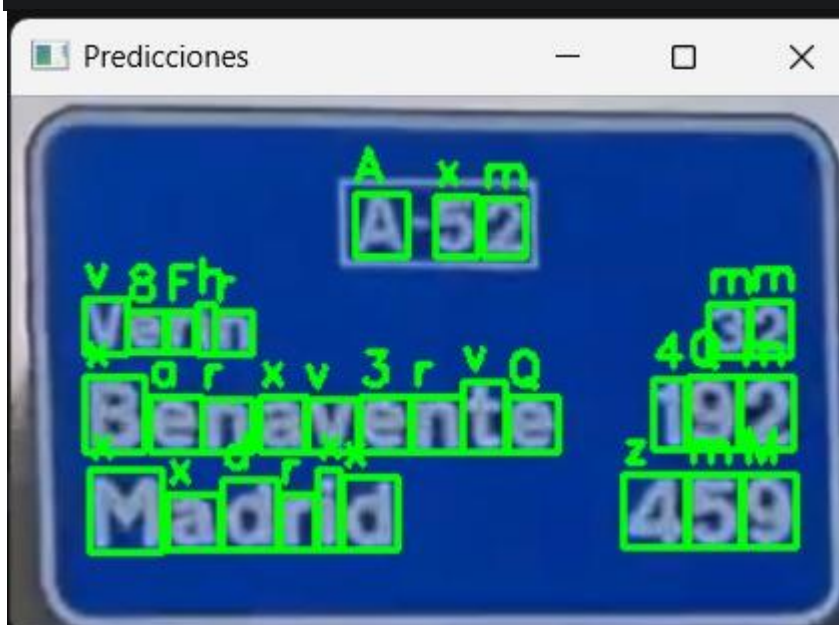


Figura 5: Imagen de predicciones

2. Encontrar los caracteres alineados

```

def ransac_lineas(boxes, umbral_y=20, min_inliers=1,
max_iter=100):
    """
    Agrupa cajas en líneas de texto usando RANSAC iterativo.
    Proceso: encuentra línea → elimina inliers → repite.

```

```

    Ordena líneas de arriba a abajo y caracteres de izquierda a
    derecha.
    """
    if len(boxes) == 0:
        return []

    restantes = list(boxes)
    lineas = []

    while len(restantes) >= min_inliers:
        mejor_inliers = []

        for _ in range(min(max_iter, len(restantes))):
            idx = np.random.randint(0, len(restantes))
            bx, by, bw, bh = restantes[idx]
            cy_ref = by + bh // 2

            inliers = [b for b in restantes
                        if abs((b[1] + b[3] // 2) - cy_ref) <
umbral_y]

            if len(inliers) > len(mejor_inliers):
                mejor_inliers = inliers

            if len(mejor_inliers) < min_inliers:
                break

            # Refinar con la media Y de los inliers
            cy_medio = np.mean([b[1] + b[3] // 2 for b in
mejor_inliers])
            inliers_final = [b for b in restantes
                             if abs((b[1] + b[3] // 2) - cy_medio)
< umbral_y]

            # Ordenar de izquierda a derecha
            lineas.append(sorted(inliers_final, key=lambda b:
b[0]))

            # Eliminar inliers de restantes
            restantes = [b for b in restantes if b not in
inliers_final]

        # Ordenar líneas de arriba a abajo
        lineas.sort(key=lambda l: np.mean([b[1] for b in l]))
    return lineas

```

3. Se ejecutará el clasificador sobre cada uno de los caracteres en el orden de izquierda a derecha y de arriba a abajo y se generara en unico string de salida.

```
def procesar_panel_ocr(img_path, clf):
```

```

img = cv2.imread(img_path)
if img is None:
    return "", 0, 0

h_orig, w_orig = img.shape[:2]

# detectar caracteres
boxes, gray, img_scaled, polaridad =
detectar_caracteres_panel(img)

# agrupar en líneas con RANSAC
# umbral_y proporcional al tamaño típico de letra
alturas = [h for (x, y, w, h) in boxes] if boxes else [20]
h_media = np.median(alturas) if alturas else 20
umbral_y = max(15, int(h_media * 0.6))
lineas = ransac_lineas(boxes, umbral_y=umbral_y,
min_inliers=1)

# clasificar en orden izquierda-derecha, arriba-abajo
texto_lineas = []
for linea in lineas:
    texto_linea = ""
    for (x, y, w, h) in linea:

        roi_bgr = img_scaled[y:y+h, x:x+w]
        if roi_bgr.size == 0:
            continue

        roi_gray = cv2.cvtColor(roi_bgr,
cv2.COLOR_BGR2GRAY)
        clahe = cv2.createCLAHE(clipLimit=2.0,
tileGridSize=(4,4))
        roi_gray = clahe.apply(roi_gray)
        if polaridad == 'norm':
            _, roi_bin = cv2.threshold(roi_gray, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        else:
            _, roi_bin = cv2.threshold(roi_gray, 0, 255,
cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
        roi_final = cv2.cvtColor(roi_bin,
cv2.COLOR_GRAY2BGR)
        pred = clf.predict(roi_final)
        char = clf.label2char(pred)
        texto_linea += char

    if texto_linea:
        texto_lineas.append(texto_linea)

```

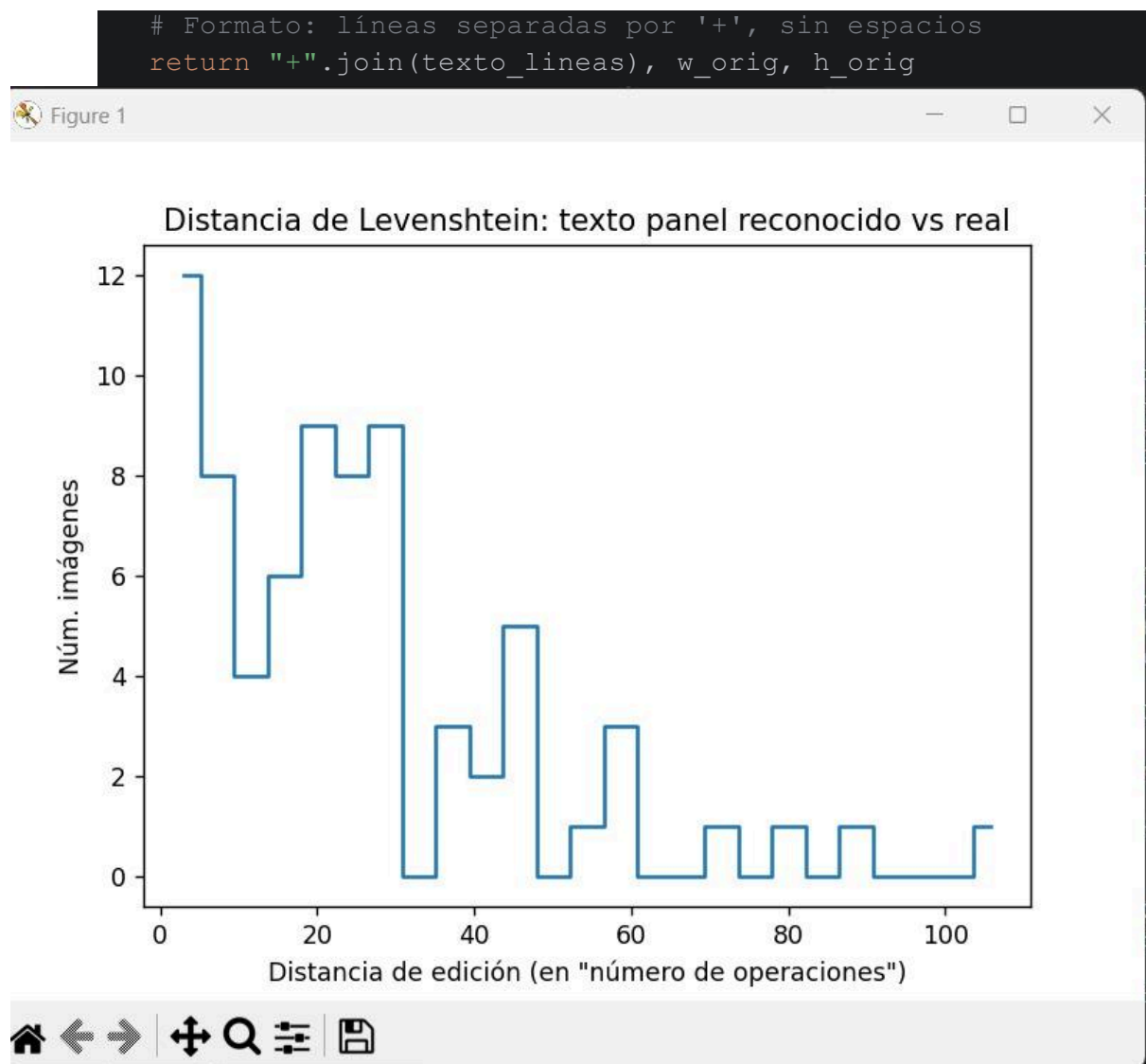


Figura 6: Histograma con distancias de Levenshtein

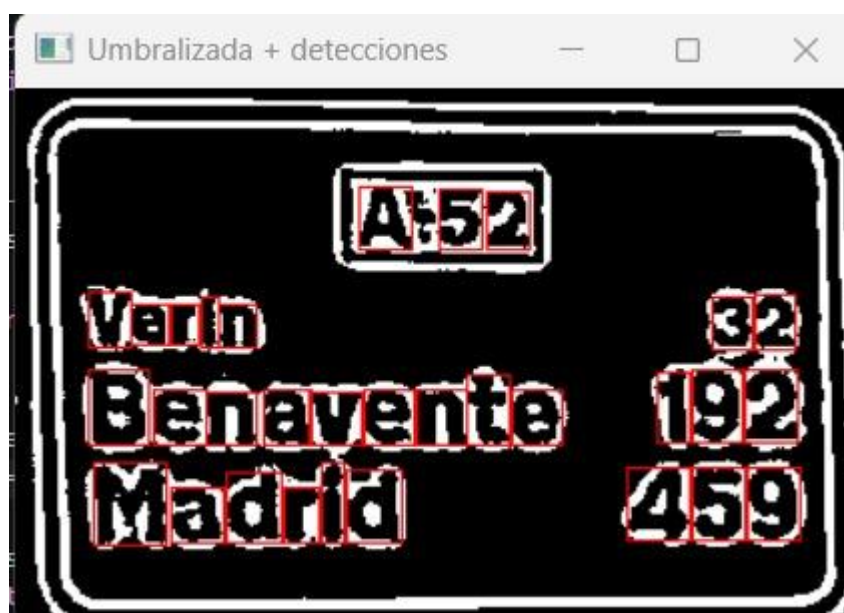


Figura 7: Umbralización y detección.

Los comandos son los siguientes:

- `python train_hog_svm.py`
 - Entrena el clasificador con los caracteres recortados de `train_ocr` y guarda el modelo en `hog_svm_model.pkl`
- `python evaluar_resultados_test_ocr_panels.py`
 - Evalúa el modelo entrenado en los paneles recortados de `test_ocr_panels` y muestra la precisión, matriz de confusión e histograma de distancia de Levenshtein
- `python text_detector_OCR.py`
 - Detecta paneles y predice caracteres de una imagen específica.
- `python ocr_classifier.py`
 - Implementa el clasificador LDA+Bayes para caracteres OCR, con métodos para entrenar, predecir y convertir entre caracteres y etiquetas.

Ejercicio 4: Lectura automática de paneles

En este ejercicio la idea es combinar la práctica 1 con la dos, para esto se han movido todos los archivos de la primera práctica a la segunda. Ahora tenemos la funcionalidad de lectura de la primera práctica con el reconocimiento de caracteres de la segunda.

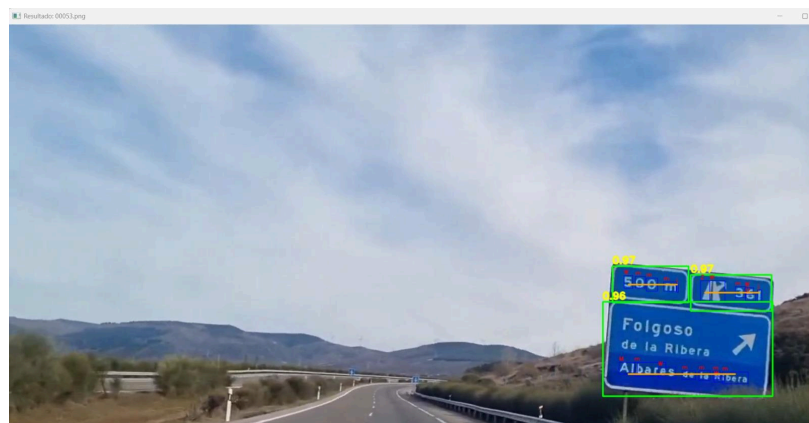


Figura 8: Lectura automática 1



Figura 9: Lectura automática 2